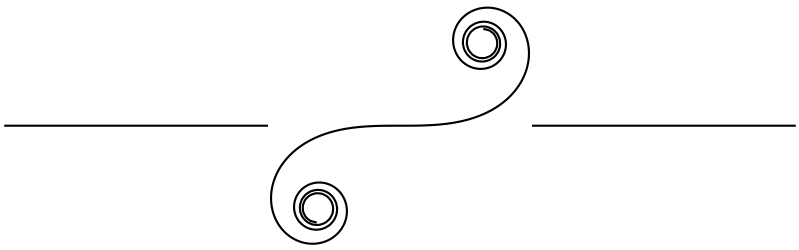
Machine Learning
Memoria de traballo

Sebastián Táboas Pazo
35679654F



USC
UNIVERSIDADE DE SANTIAGO DE COMPOSTELA
FACULTADE DE FÍSICA

Data: 9 de novembro do 2026

Pódese revisar o repositorio do traballo con todos os seus respectivos commits e moitos máis resultados intermedios no seguinte enlace:
`https://github.com/sebastian-taboas/Biophysics/tree/main/traballo1_redes_neuronais`

Índice de contidos

0. Introdución

1. Depuración de datos

2. Representacións estatísticas

3. Análise con sklearn (regresión loxística)

    3.0.1 Modelo Linear

    3.0.2 Modelo RBF

  3.1 Análise sen sesgo

    3.1.1 Estimación, training e resultados do Modelo Linear/Loxístico linear

    3.1.2 Estimación, training e resultados do Modelo RBF

    3.1.3 Estimación, training e resultados do Modelo RandomForest

    3.1.4 Estimación, training e resultados do Modelo HistGradientBoosting

  3.2 Conclusións

  3.3 Análise de regresións (Score)

4. Análise con redes neuronais

5. Conclusións finais

Apéndice Cross Validation en redes neuronais

Resumo

O noso obxectivo neste traballo foi predicir resultados dun dataframe mediante métodos de machine learning. Primeiro limpiamos un pouco os nosos datos e facemos e os adaptamos para podelos procesor. Nun segundo paso estudamos un pouco como son esos datos aínda que non nos paramos realmente a facer ningunha clase de estudo estatístico. Despois empregamos funcións da librería de sklearn para obter predicións. Nun primer momento aplicamos o modelo máis sinxelo, unha regresión linear que optimizamos os seus parámetros cunha cross validation pero máis desglosada e mostrando as

vísceras do cálculo; facelo así supón unha maior carga computacional pero vese mellor o que fan os programas de cross validation que usaremos máis adiante con GridSearchCV ou keras_tuner.RandomSearch. Os resultados da recta axustada son aínda bastante pésimos, empregamos tamén un modelo RBF (Radial BasisFunction) que considera no linearidades mellorando increíblemente o sistema. Como atopamos que este segundo ten unha predición altísima consideramos que hai algunha das nosas variables que está moi correlacionada coa que queremos determinar aplicando mutual_info_classif.

Volvemos a crear modelos pero esta vez antes de aplicalos realizamos validación cruzada coa función GridSearchCV para optimizar os parámetros dos modelos. Nun primeiro lugar repetimos co modelo linear (loss = hinge) que xa vemos que non nos dá moi bos resultados e o comparamos coa regresión loxística (loss = log_loss) e xa por aforrar só vemos que tal procesa a regresión loxística. Aplicamos cv sobre a rbf tamén e seguimos obtendo moi bos resultados (de feito será a técnica que mellor prediga os nosos resultados durante todo o traballo). Engadimos agora dous modelos máis sobre os que tamén facemos cv: RandomForest e HistGradientBoosting. O primeiro dá resultados mellores que a regresión loxística pero cae 5 puntos detrás do rbf e ademais ten overfitting (isto vese en que a accuracy_score para os datos de entrenamento ronda o valor da unidade). O segundo modelo mellora respecto do anterior e, aínda que menos, segue tendo bastante overfitting.

Agora toca o momento de usar keras e crear redes neuronais, aínda que breve, é a sección que máis horas ten detrás. Lendo documentación das funcións de activación das neuronas e moitas outras funcións que incorpora keras nas súas librarías pretendimos crear unha rede óptima, i.e., que con poucas iteracións (epochs) no entrenamento conseguiramos resultados moi bos. En xeral as nosas redes saturaban a precisión sobre os 80 puntos, non se conseguiu melloralo nin tampouco entendo moi ben por que satura. A gran maioría de redes simples empezaban a memorizar os datos baixando moito a precisión no teste e senón a función de loss mantiña valores bastante altos e algo inestables, a solució para isto foi implementar unha penalización sobre os pesos na función de activación: implementamos L2 de keras.regularizers para matar o crecemento continuado dos pesos nos parámetros ao entrenar moitas neuronas. A elección dos núcleos dos pesos é un pouco por conveniencia, na bibliografía recoméndase utilizar os que aparecen para cada función de activación. Un valor importante era o optimizador, empregar activadores basados en RMSprop (Adam e derivados) por algunha razón, aínda que conseguían valores lixeiramente mellores como velocidade ou precisión, aparecía un ruído moi significativo nas funcións de loss e de accuracy, polo que a nosa mellor opción foi empregar SGD sen momento (aplicar nesterov produce en xeral diverxencias por enriba das 60 epochs). Probouse tamén a poñer distintos valores de learning rate, incluso implemetouse vía keras.optimizers.schedules learnings rates dinámicos que se axusten mellor nas fases de aprendizaxe pero non melloraba a cousa. Finalmente, practicamente calquera rede con case calquera función de activación comportábase ben, só había dous problemas: as que funcionaban rápido presentaban overfit antes de acadar a saturación dos 80 puntos na precisión e senón eran considerablemente máis lentas; a diferencia recaía na cantidade de units (neuronas) das capas que incrementa os parámetros de

axuste segundo

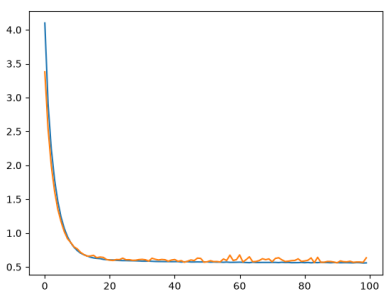
$$n^{j}_{\text{parametros}} = (units_{j-1} + 1) * units_{j}.$$

Escollimos a primeira vía, as rápidas pero overfiteadas pero resolveuse engadindo capas layers.Dropout().

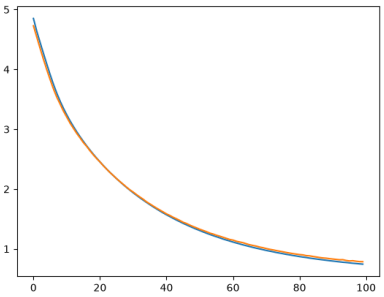
Algúns resultados da optimización de redes

Fixéronse moitas probas de testeo a machete para comparar parámetros con moitas funcións de activación, entre elas unha rede con 2 capas Dense(relu) e unha Dense(softplus) ademais da de input e a de saída con softmax que podemos visualizar as funcións de loss máis abaixo (coas que estudamos se ocorría overfitting ou underfitting ademais da estabilidade). As imaxes están codificados segundo a táboa de abaixo.

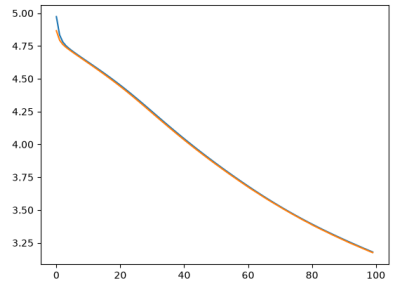
activate function	optimizer	learning rate	batch size
	x1xx = SGD		
1xxx = softplus postInput	x2xx = SGD + nesterov	xx1x = 0.01	xxx1 = 16
2xxx = softplus no medio	x3xx = Adam	xx2x = 0.001	xxx2 = 8
	x4xx = Nadam	xx3x = 0.0001	
	x5xx = AdamW		



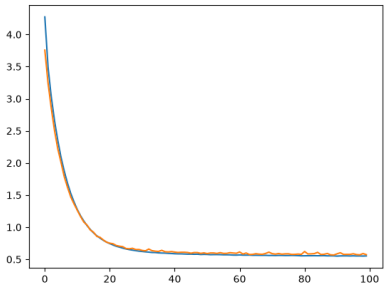
(a) 1111



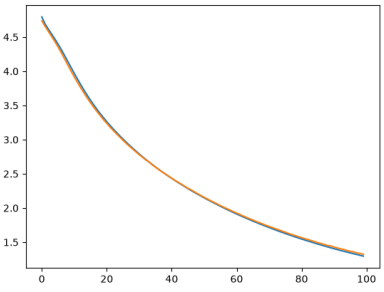
(b) 1121



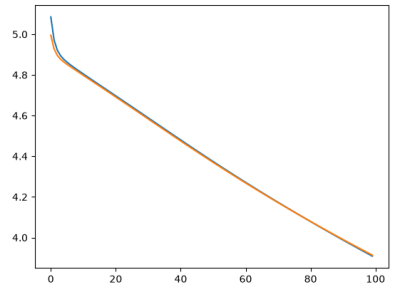
(c) 1131



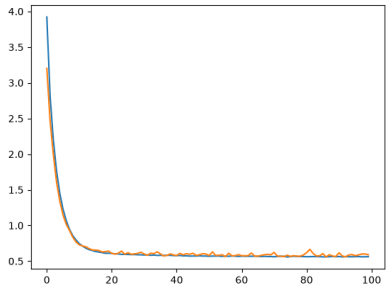
(d) 1112



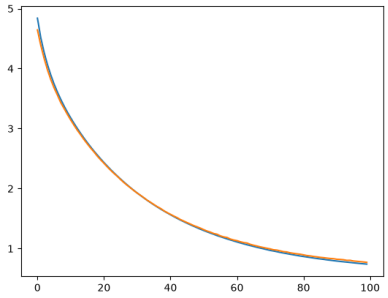
(e) 1122



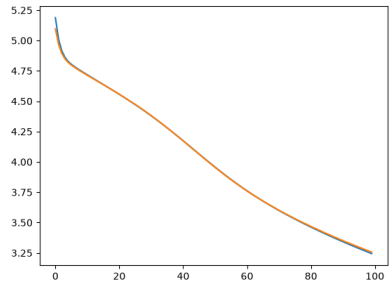
(f) 1132



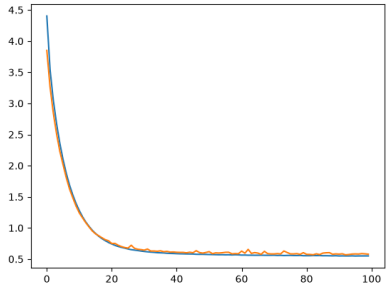
(a) 1211



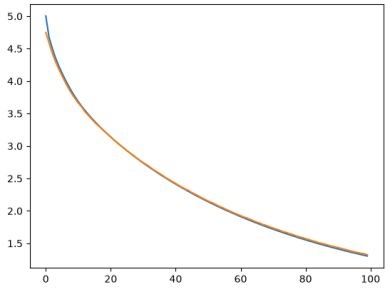
(b) 1221



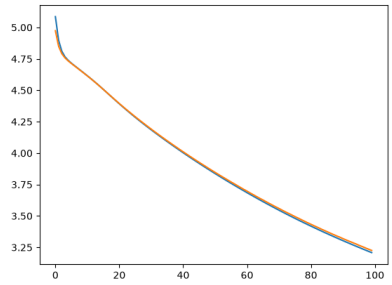
(c) 1231



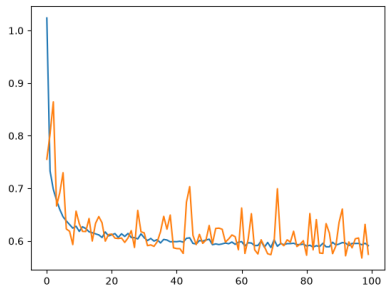
(d) 1212



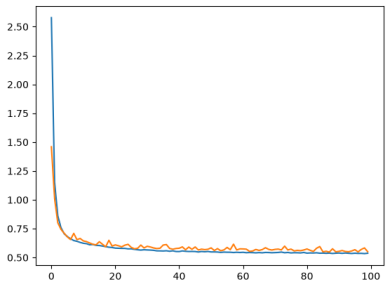
(e) 1222



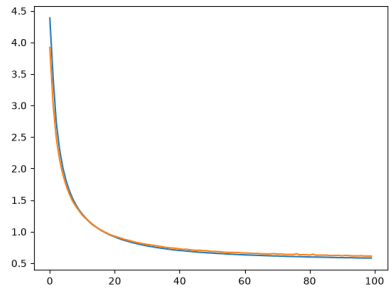
(f) 1232



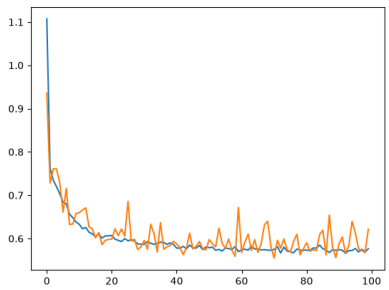
(a) 1311



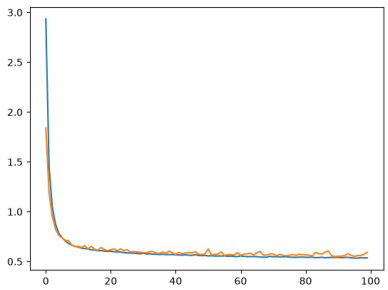
(b) 1321



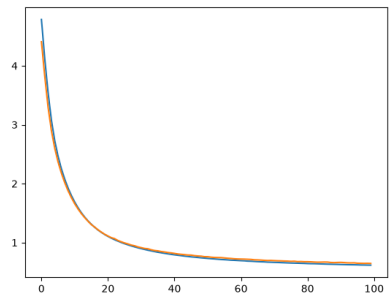
(c) 1331



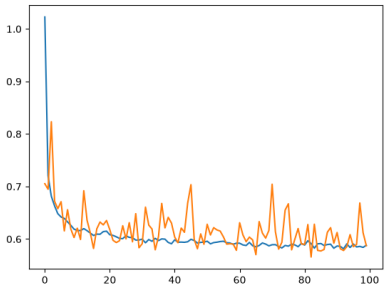
(d) 1312



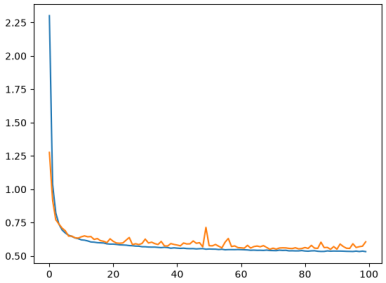
(e) 1322



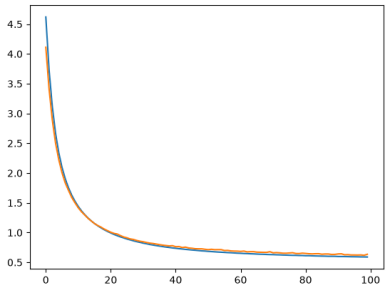
(f) 1332



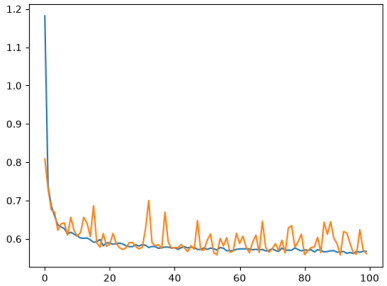
(a) 1411



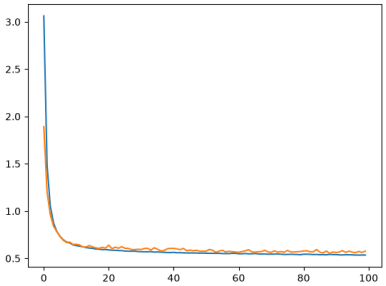
(b) 1421



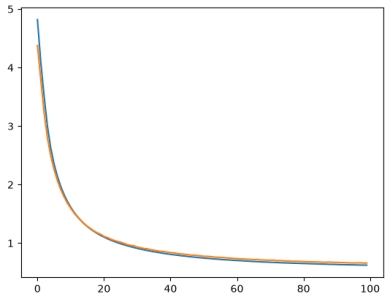
(c) 1431



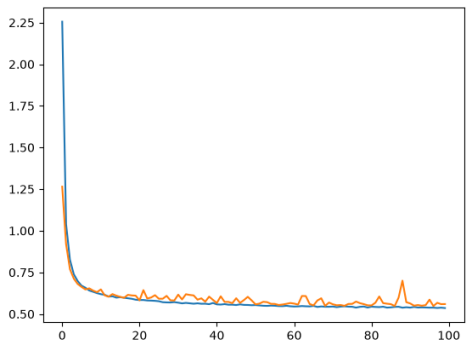
(d) 1412



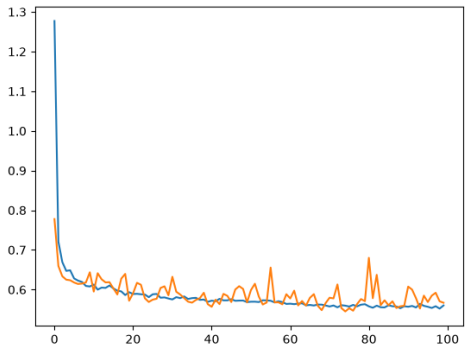
(e) 1422



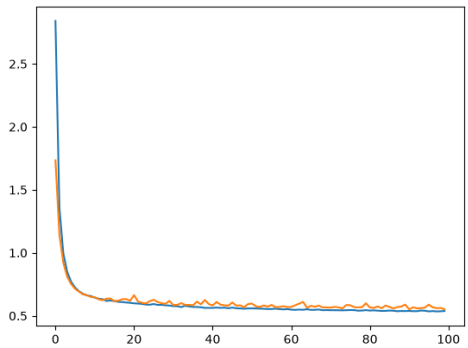
(f) 1432



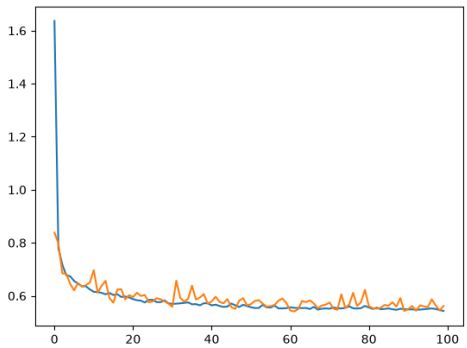
(a) 1511



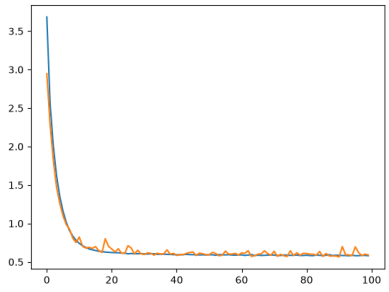
(b) 1521



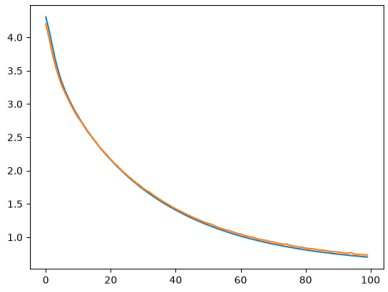
(c) 1512



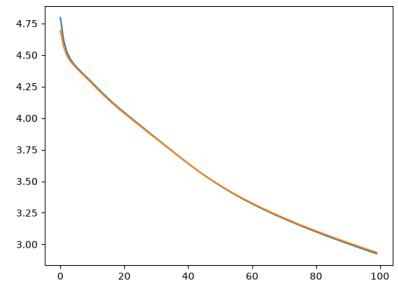
(d) 1522



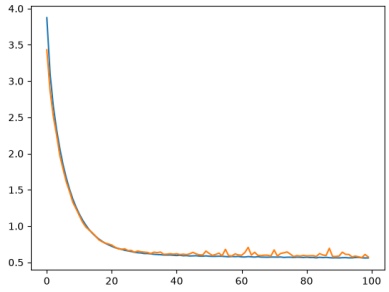
(a) 2111



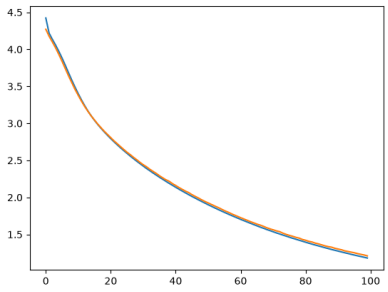
(b) 2121



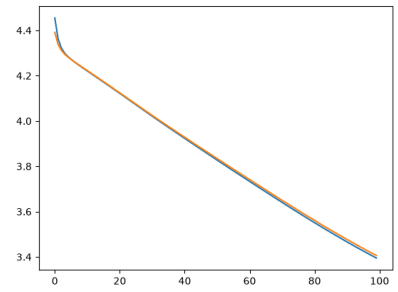
(c) 2131



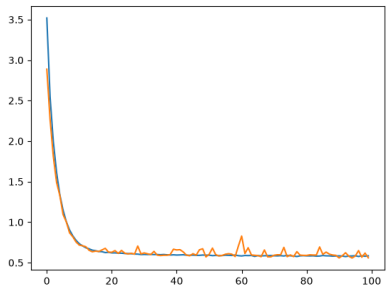
(d) 2112



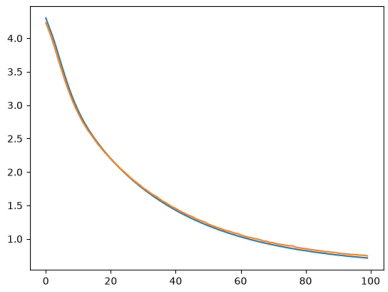
(e) 2122



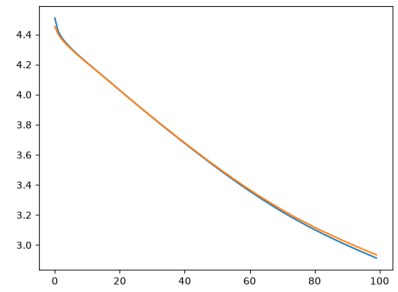
(f) 2132



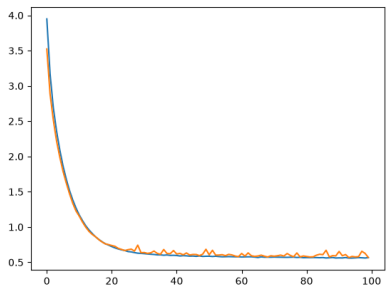
(a) 2211



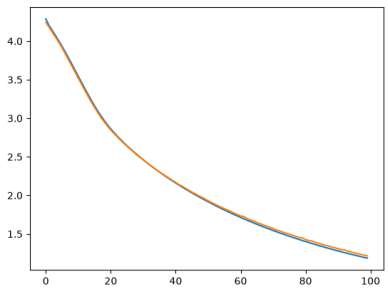
(b) 2221



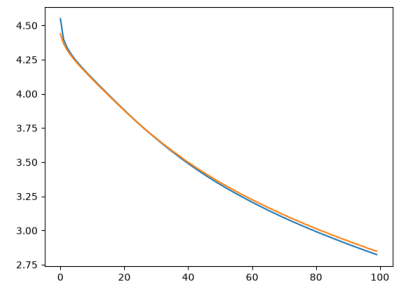
(c) 2231



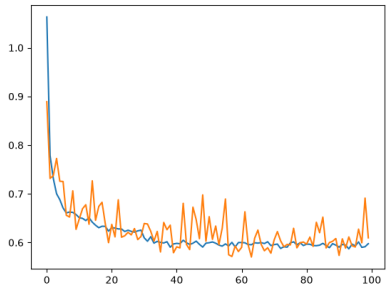
(d) 2212



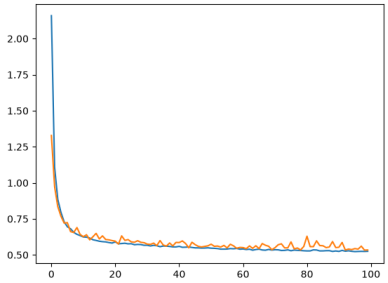
(e) 2222



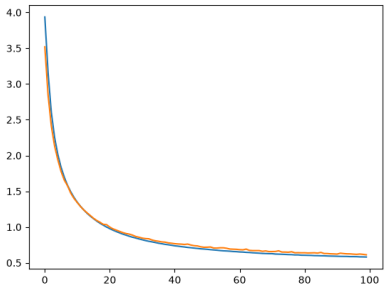
(f) 2232



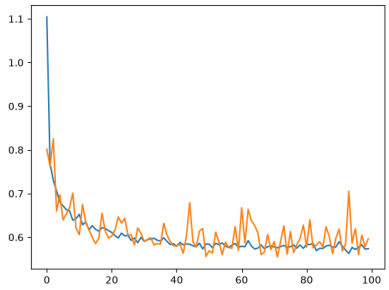
(a) 2311



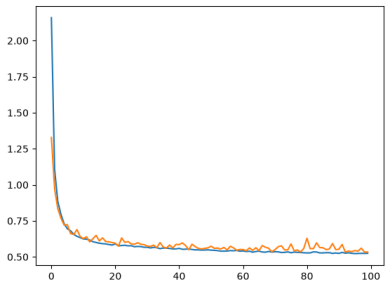
(b) 2321



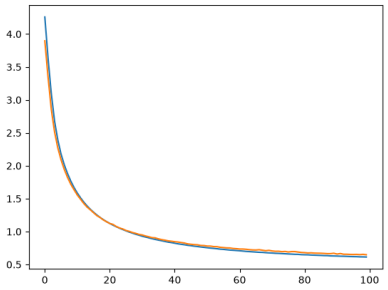
(c) 2331



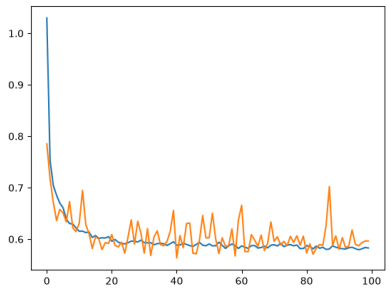
(d) 2312



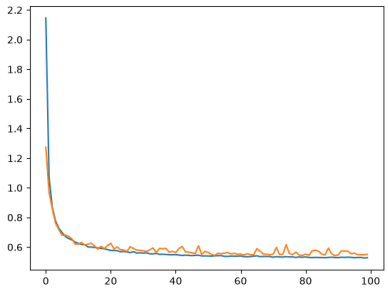
(e) 2322



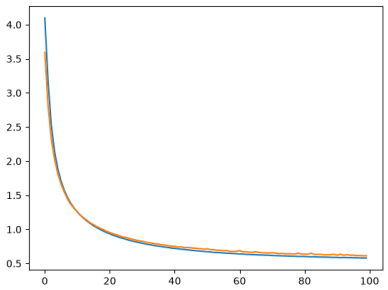
(f) 2332



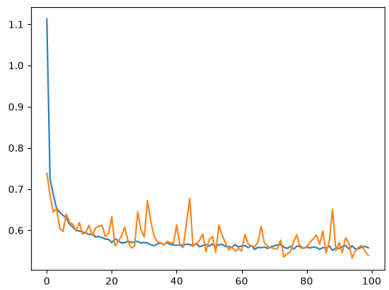
(a) 2411



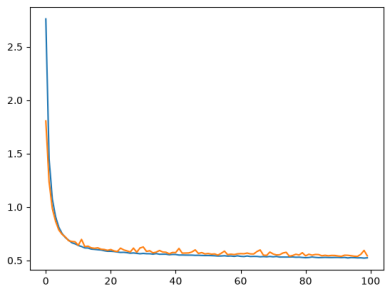
(b) 2421



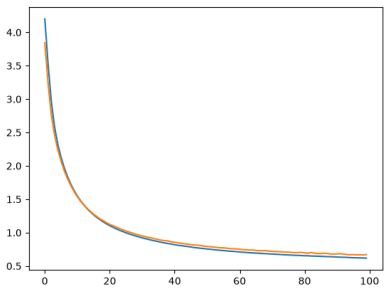
(c) 2431



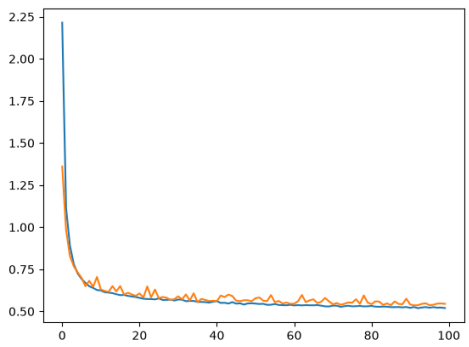
(d) 2412



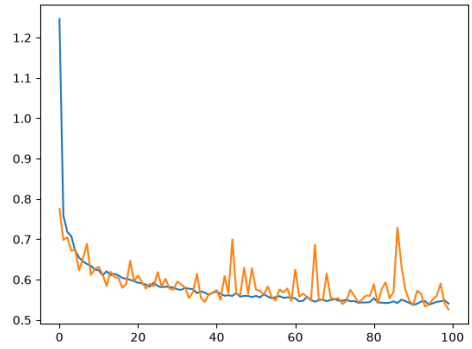
(e) 2422



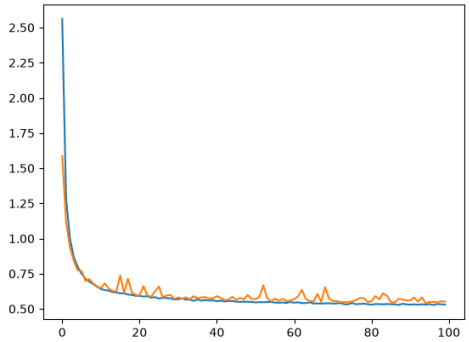
(f) 2432



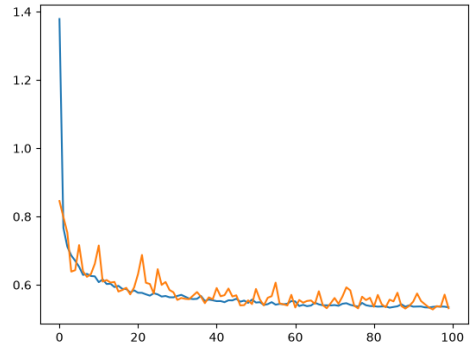
(a) 2511



(b) 2521

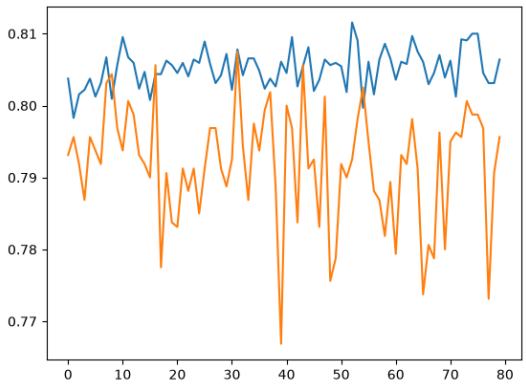


(c) 2512

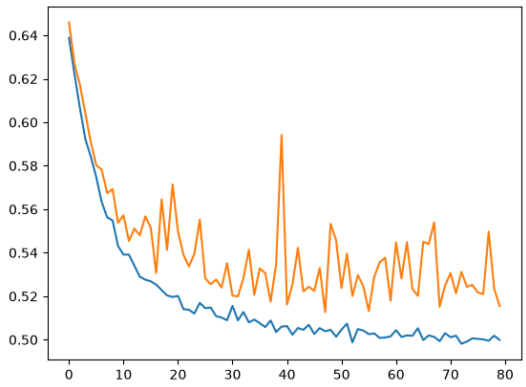


(d) 2522

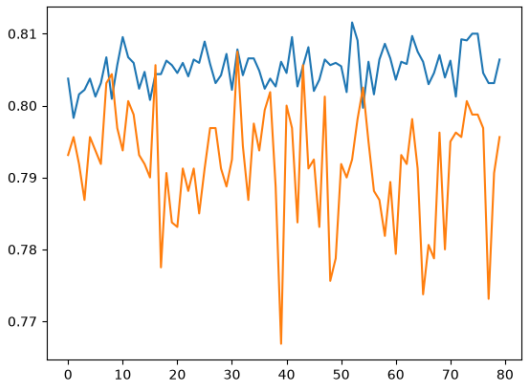
Ademais disto cando conseguimos unhas redes que predicían bastante ben aplicamos un cv sobre elas para optimizar os parámetros. Esiximos 100 probas distintas de parámetros sobre a nosa rede, obviamente había moitas combinacións que causarían moito ruído pero non se soubo excluír estas do programa e aínda así resulta que aquelas que maximizaban a precisión da rede (moitas veces por pura aleatoriedade xa que estaba chea de ruído) tiña diferenzas infímas frente a outras combinacións estables; entre val_accuracy = 0.79 aos 0.80 dunha estable as que mellor acertaban tiñan o máximo en 0.808. Este proceso que levou un total de 7 horas e 20 minutos de compilación foi probado co que chamamos modelo_simple e modelo_mellorada no notebook, os resultados gráficos da comparación de accuracy vs val_accuracy e loss vs val_loss son



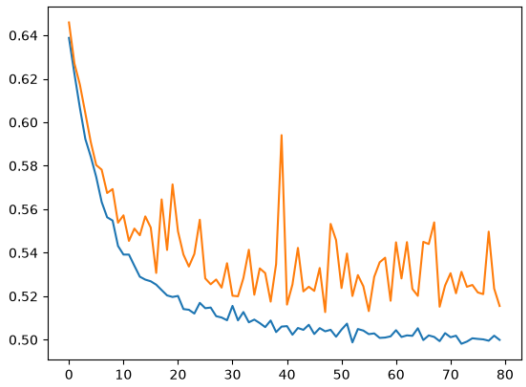
(a) Precisión rede simple



(b) Loss rede simple

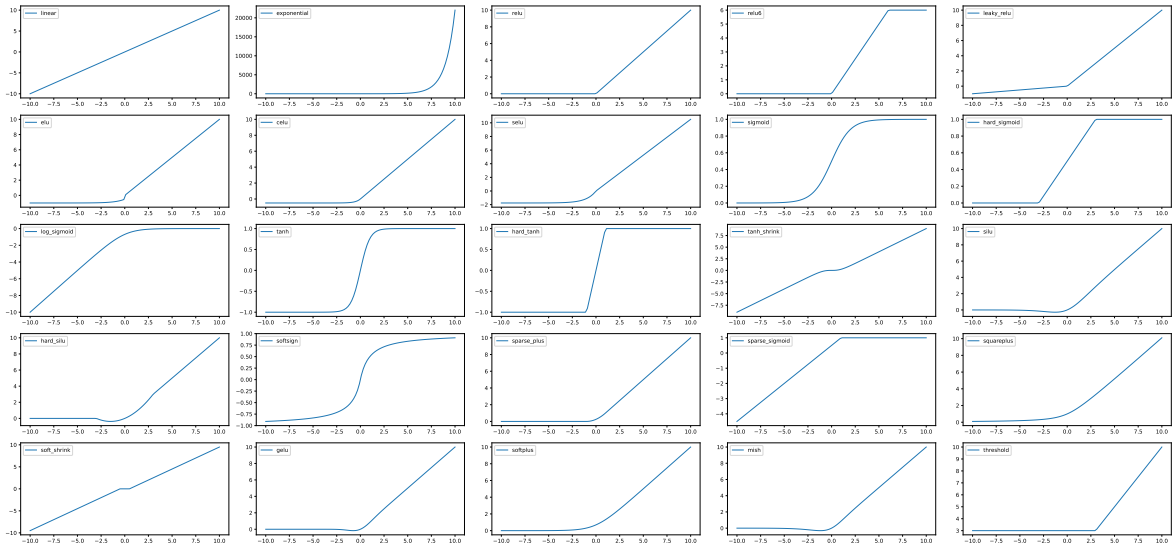


(c) Precisión rede mellorada



(d) Loss rede mellorada

Ademais disto fixeronse moitas probas individuais con outros activadores como *mish*, *silu*, *relu*, *celu*, *leaky_relu*, *elu* e *softplus*. A modo anecdótico a representación gráfica de case todas as funcións de activación posibles dunha neuronal son as seguintes



Por último, os nosos intentos de atopar unha rede neuronal estable e con bastante poder predictivo e que sexa computacionalmente menos custosasç levounos ao seguinte modelo

```
modelo_optimizado = Sequential([
    layers.Input(shape=(data.shape[1],)),
    layers.Dense(128, activation='silu',
        kernel_initializer='glorot_uniform',
        kernel_regularizer=L1L2(l1=0.0002, l2=0.01)),
    layers.Dropout(0.1),
    layers.Dense(16, activation='celu',
        kernel_initializer='he_normal',
        kernel_regularizer=L2(0.01)),
    layers.Dense(40, activation='celu',
        kernel_initializer='he_normal',
        kernel_regularizer=L2(0.01)),
    layers.Dropout(0.1),
    layers.Dense(16, activation='relu',
        kernel_initializer='he_normal',
        kernel_regularizer=L2(0.01)),
    layers.Dense(4, activation='softmax') ])

modelo_opmizado.compile(
    optimizer=SGD(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'] )

modelo_optmizado.fit(
    x2_train_scaled, y2_train_enc,
    epochs=70, batch_size=8,
    validation_data=(x2_test_scaled, y2_test_enc))
```

que comparamos con outros dous modelos máis simples, os resultados gráficos son os seguintes

